```
!pip install yfinance gspread oauth2client pytz sqlalchemy -q
print("✅ All libraries installed")
```

```
✅ All libraries installed
```

```
import yfinance as yf
import pandas as pd
import gspread
import pytz
import os
from sqlalchemy import create_engine, text
from datetime import datetime
from google.colab import auth, drive
from google.auth import default

# Authenticate Google account
auth.authenticate_user()
creds, _ = default()
gc = gspread.authorize(creds)

# Mount Google Drive
drive.mount('/content/drive')

IST         = pytz.timezone('Asia/Kolkata')
SHEET_NAME  = "Jio Finance Stock Analysis"
DB_FOLDER   = '/content/drive/MyDrive/JioFinance'
DB_PATH     = f'{DB_FOLDER}/jiofin_stock.db'
TICKER      = "JIOFIN.NS"

os.makedirs(DB_FOLDER, exist_ok=True)
print(f"✅ Auth complete | Drive mounted | DB folder ready")
print(f"🕐 Current IST time: {datetime.now(IST).strftime('%Y-%m-%d %H:%M:%S')}")
```

```
Mounted at /content/drive
✅ Auth complete | Drive mounted | DB folder ready
🕐 Current IST time: 2026-04-25 19:39:24
```

```
engine = create_engine(f'sqlite:///{DB_PATH}')

create_table_sql = """
CREATE TABLE IF NOT EXISTS jiofin_daily (
    id               INTEGER PRIMARY KEY AUTOINCREMENT,
    date             DATE UNIQUE,
    ticker           TEXT,
    open_price       REAL,
    high_price       REAL,
    low_price        REAL,
    close_price      REAL,
    volume           INTEGER,
    daily_return_pct REAL,
    price_range      REAL,
    moving_avg_7     REAL,
    moving_avg_30    REAL,
```

```
    moving_avg_50    REAL,
    fetched_at       TEXT
);
"""

with engine.connect() as conn:
    conn.execute(text(create_table_sql))
    conn.commit()

print(f"✅ SQLite database created at: {DB_PATH}")
```

```
✅ SQLite database created at: /content/drive/MyDrive/JioFinance/jiofin_stock.db
```

```
def fetch_jiofin_data(period="1y", interval="1d"):
    try:
        print(f"  🚀 Fetching {TICKER} data...")
        stock = yf.Ticker(TICKER)
        df    = stock.history(period=period, interval=interval)

        if df.empty:
            print("  ⚠️  No data returned — market may be closed")
            return None

        df.reset_index(inplace=True)

        # Fix timezone-aware date column
        if hasattr(df['Date'].dtype, 'tz') and df['Date'].dtype.tz is not None:
            df['Date'] = df['Date'].dt.tz_localize(None)
        df['Date'] = pd.to_datetime(df['Date']).dt.date

        df['Ticker']     = 'JIOFIN'
        df['Fetched_At'] = datetime.now(IST).strftime('%Y-%m-%d %H:%M:%S')

        df.rename(columns={
            'Open'  : 'open_price',
            'High'  : 'high_price',
            'Low'   : 'low_price',
            'Close' : 'close_price',
            'Volume': 'volume',
            'Date'  : 'date'
        }, inplace=True)

        df['daily_return_pct'] = df['close_price'].pct_change() * 100
        df['price_range']      = df['high_price'] - df['low_price']
        df['moving_avg_7']     = df['close_price'].rolling(7).mean()
        df['moving_avg_30']    = df['close_price'].rolling(30).mean()
        df['moving_avg_50']    = df['close_price'].rolling(50).mean()

        df = df[['date','Ticker','open_price','high_price','low_price',
                 'close_price','volume','daily_return_pct','price_range',
                 'moving_avg_7','moving_avg_30','moving_avg_50','Fetched_At']]

        df.columns = ['date','ticker','open_price','high_price','low_price',
                      'close_price','volume','daily_return_pct','price_range',
                      'moving_avg_7','moving_avg_30','moving_avg_50','fetched_at']
```

```
            print(f"  ✅ Fetched {len(df)} rows")
            print(f"  🗓️ Latest date  : {df['date'].max()}")
            print(f"  💰 Latest close : ₹{df['close_price'].iloc[-1]:.2f}")
            return df

        except Exception as e:
            print(f"  ❌ Fetch error: {e}")
            return None

    print("✅ fetch_jiofin_data() function ready")
```

```
✅ fetch_jiofin_data() function ready
```

```
def upsert_to_sql(df, engine):
    try:
        with engine.connect() as conn:
            for _, row in df.iterrows():
                conn.execute(text("""
                    INSERT OR REPLACE INTO jiofin_daily
                    (date, ticker, open_price, high_price, low_price,
                     close_price, volume, daily_return_pct, price_range,
                     moving_avg_7, moving_avg_30, moving_avg_50, fetched_at)
                    VALUES
                    (:date, :ticker, :open_price, :high_price, :low_price,
                     :close_price, :volume, :daily_return_pct, :price_range,
                     :moving_avg_7, :moving_avg_30, :moving_avg_50, :fetched_at)
                """), {
                    'date'            : str(row['date']),
                    'ticker'          : str(row['ticker']),
                    'open_price'      : round(float(row['open_price']), 2),
                    'high_price'      : round(float(row['high_price']), 2),
                    'low_price'       : round(float(row['low_price']), 2),
                    'close_price'     : round(float(row['close_price']), 2),
                    'volume'          : int(row['volume']),
                    'daily_return_pct': round(float(row['daily_return_pct']), 4)
                                        if pd.notna(row['daily_return_pct']) else None,
                    'price_range'     : round(float(row['price_range']), 2),
                    'moving_avg_7'    : round(float(row['moving_avg_7']), 2)
                                        if pd.notna(row['moving_avg_7']) else None,
                    'moving_avg_30'   : round(float(row['moving_avg_30']), 2)
                                        if pd.notna(row['moving_avg_30']) else None,
                    'moving_avg_50'   : round(float(row['moving_avg_50']), 2)
                                        if pd.notna(row['moving_avg_50']) else None,
                    'fetched_at'      : str(row['fetched_at'])
                })
            conn.commit()

        total = pd.read_sql(
            "SELECT COUNT(*) as cnt FROM jiofin_daily", engine)['cnt'][0]
        print(f"  ✅ SQL updated — {total} total rows in database")

    except Exception as e:
        print(f"  ❌ SQL error: {e}")

print("✅ upsert_to_sql() function ready")
```

```
✅ upsert_to_sql() function ready
```

```
def push_to_google_sheets(engine, gc, sheet_name):
    try:
        try:
            sh = gc.open(sheet_name)
        except gspread.SpreadsheetNotFound:
            sh = gc.create(sheet_name)
            sh.share('', perm_type='anyone', role='reader')
            print(f"  📄 New sheet created: {sh.url}")

        # ── Tab 1: Daily Data ──────────────────────────
        raw_df = pd.read_sql(
            "SELECT * FROM jiofin_daily ORDER BY date DESC", engine)
        raw_df = raw_df.fillna('').astype(str)

        try:
            ws1 = sh.worksheet("Daily_Data")
        except:
            ws1 = sh.add_worksheet("Daily_Data", rows=2000, cols=20)
        ws1.clear()
        ws1.update([raw_df.columns.tolist()] + raw_df.values.tolist())
        print(f"  ✅ Daily_Data tab — {len(raw_df)} rows written")

        # ── Tab 2: Monthly Summary ─────────────────────
        monthly_df = pd.read_sql("""
            SELECT
                strftime('%Y-%m', date)        AS month,
                ROUND(AVG(close_price), 2)     AS avg_close,
                MAX(high_price)                AS month_high,
                MIN(low_price)                 AS month_low,
                SUM(volume)                    AS total_volume,
                ROUND(SUM(daily_return_pct),2) AS total_return_pct
            FROM jiofin_daily
            GROUP BY month
            ORDER BY month DESC
        """, engine).fillna('').astype(str)

        try:
            ws2 = sh.worksheet("Monthly_Summary")
        except:
            ws2 = sh.add_worksheet("Monthly_Summary", rows=100, cols=10)
        ws2.clear()
        ws2.update([monthly_df.columns.tolist()] + monthly_df.values.tolist())
        print(f"  ✅ Monthly_Summary tab — {len(monthly_df)} months written")

        # ── Tab 3: Live Snapshot ───────────────────────
        try:
            info = yf.Ticker(TICKER).info
        except:
            info = {}

        snapshot = [
            ["Metric",         "Value"],
            ["Current Price",  info.get("currentPrice",       "N/A")],
```

```
            ["52W High",       info.get("fiftyTwoWeekHigh",   "N/A")],
            ["52W Low",        info.get("fiftyTwoWeekLow",    "N/A")],
            ["Market Cap",     info.get("marketCap",          "N/A")],
            ["P/E Ratio",      info.get("trailingPE",         "N/A")],
            ["Volume",         info.get("volume",             "N/A")],
            ["Avg Volume",     info.get("averageVolume",      "N/A")],
            ["Beta",           info.get("beta",               "N/A")],
            ["Last Updated",   datetime.now(IST).strftime('%Y-%m-%d %H:%M:%S')]
        ]

        try:
            ws3 = sh.worksheet("Live_Snapshot")
        except:
            ws3 = sh.add_worksheet("Live_Snapshot", rows=20, cols=5)
        ws3.clear()
        ws3.update(snapshot)
        print(f"  ✅ Live_Snapshot tab updated")

        print(f"\n  🔗 Sheet URL: {sh.url}")

    except Exception as e:
        print(f"  ❌ Sheets error: {e}")

print("✅ push_to_google_sheets() function ready")
```

```
✅ push_to_google_sheets() function ready
```

```
def write_timestamp(gc, sheet_name, df, run_count):
    try:
        sh  = gc.open(sheet_name)
        try:
            ws = sh.worksheet("Last_Updated")
        except:
            ws = sh.add_worksheet("Last_Updated", rows=15, cols=5)

        now = datetime.now(IST).strftime('%Y-%m-%d %H:%M:%S')
        ws.clear()
        ws.update([
            ["Status",       "Value"],
            ["Last Run",     now],
            ["Run Count",    run_count],
            ["Total Rows",   len(df)],
            ["Latest Date",  str(df['date'].max())],
            ["Latest Close", f"₹{df['close_price'].iloc[-1]:.2f}"],
            ["Pipeline",     "LIVE - Auto updating"]
        ])
        print(f"  ✅ Timestamp written: {now}")

    except Exception as e:
        print(f"  ❌ Timestamp error: {e}")

print("✅ write_timestamp() function ready")
```

```
✅ write_timestamp() function ready
```

```
def check_status(engine, gc, sheet_name):
    print("\n" + "="*48)
    print("     JIOFIN PIPELINE — STATUS CHECK")
    print("="*48)

    # SQL check
    try:
        total  = pd.read_sql(
            "SELECT COUNT(*) as c FROM jiofin_daily", engine)['c'][0]
        latest = pd.read_sql(
            "SELECT MAX(date) as d FROM jiofin_daily", engine)['d'][0]
        price  = pd.read_sql(
            "SELECT close_price FROM jiofin_daily ORDER BY date DESC LIMIT 1",
            engine)['close_price'][0]
        print(f"\n  SQL DATABASE")
        print(f"  Total rows   : {total}")
        print(f"  Latest date  : {latest}")
        print(f"  Latest close : ₹{price:.2f}")
    except Exception as e:
        print(f"  SQL error: {e}")

    # Sheets check
    try:
        sh   = gc.open(sheet_name)
        ws   = sh.worksheet("Daily_Data")
        rows = len(ws.get_all_values()) - 1
        print(f"\n  GOOGLE SHEETS")
        print(f"  Sheet rows   : {rows}")
        print(f"  URL          : {sh.url}")
    except Exception as e:
        print(f"  Sheets error : {e}")

    # Live price check
    try:
        live  = yf.Ticker(TICKER).info
        price = live.get('currentPrice', 'Market closed')
        print(f"\n  LIVE MARKET")
        print(f"  Current price: ₹{price}")
    except:
        print(f"  Live price   : unavailable")

    print(f"\n  Checked at   : {datetime.now(IST).strftime('%Y-%m-%d %H:%M:%S')} IST")
    print("="*48)

print("✅ check_status() function ready")
```

```
✅ check_status() function ready
```

```
# ═══════════════════════════════════════════════
#   RUN THIS CELL ONCE DAILY AFTER 3:30 PM IST
#   It fetches, saves SQL, updates Google Sheets
# ═══════════════════════════════════════════════

run_count = 0

def run_pipeline():
```

```
    global run_count
    run_count += 1
    now = datetime.now(IST)

    print(f"\n{'='*50}")
    print(f"  RUN #{run_count} | {now.strftime('%Y-%m-%d %H:%M:%S')} IST")
    print(f"{'='*50}")

    try:
        # Step 1 — Fetch
        print("\n  [1/4] Fetching JIOFIN data...")
        df = fetch_jiofin_data(period="1y", interval="1d")
        if df is None:
            print("  ❌ Skipping — no data returned")
            return

        # Step 2 — Save SQL
        print("\n  [2/4] Saving to SQL database...")
        upsert_to_sql(df, engine)

        # Step 3 — Push to Sheets
        print("\n  [3/4] Pushing to Google Sheets...")
        push_to_google_sheets(engine, gc, SHEET_NAME)

        # Step 4 — Timestamp
        print("\n  [4/4] Writing timestamp...")
        write_timestamp(gc, SHEET_NAME, df, run_count)

        print(f"\n  ✅ PIPELINE COMPLETE!")
        print(f"  All 4 steps done successfully")

    except Exception as e:
        print(f"\n  ❌ Pipeline error: {e}")

# Run it now
run_pipeline()
```

```
==================================================
  RUN #1 | 2026-04-25 19:40:12 IST
==================================================

  [1/4] Fetching JIOFIN data...
  📡 Fetching JIOFIN.NS data...
  ✅ Fetched 248 rows
  🗓️ Latest date  : 2026-04-24
  💰 Latest close : ₹245.73

  [2/4] Saving to SQL database...
  ✅ SQL updated — 249 total rows in database

  [3/4] Pushing to Google Sheets...
  ✅ Daily_Data tab — 249 rows written
  ✅ Monthly_Summary tab — 13 months written
  ✅ Live_Snapshot tab updated

  🔗 Sheet URL: https://docs.google.com/spreadsheets/d/1AzJFoYeDsyVFQFGdBBA1sBFM4HVbG3EniyxPEFjBMBU
```

```
  [4/4] Writing timestamp...
  ✅ Timestamp written: 2026-04-25 19:40:20

  ✅ PIPELINE COMPLETE!
  All 4 steps done successfully
```

```
# Run this after Cell 9 to confirm all data is correct
check_status(engine, gc, SHEET_NAME)
```

```
================================================
     JIOFIN PIPELINE — STATUS CHECK
================================================

  SQL DATABASE
  Total rows   : 249
  Latest date  : 2026-04-24
  Latest close : ₹245.73

  GOOGLE SHEETS
  Sheet rows   : 249
  URL          : https://docs.google.com/spreadsheets/d/1AzJFoYeDsyVFQFGdBBA1sBFM4HVbG3EniyxPEFjBMBU

  LIVE MARKET
  Current price: ₹245.73

  Checked at   : 2026-04-25 19:40:25 IST
================================================
```

```
import time
import pytz
from datetime import datetime

IST          = pytz.timezone('Asia/Kolkata')
INTERVAL_SEC = 300   # ← change this: 300=5min, 900=15min, 3600=1hr, 86400=24hr

run_count    = 0

def is_market_open():
    now  = datetime.now(IST)
    mins = now.hour * 60 + now.minute
    day  = now.weekday()          # 0=Mon … 6=Sun
    if day >= 5:
        return False, "Weekend — NSE closed"
    if mins < 9 * 60 + 15:
        return False, "Pre-market — opens 9:15 AM IST"
    if mins > 15 * 60 + 30:
        return False, "After-hours — closed 3:30 PM IST"
    return True, "Market OPEN"

def run_once():
    global run_count
    run_count += 1
    now              = datetime.now(IST)
    market_open, msg = is_market_open()

    print(f"\n{'═'*52}")
    print(f"  RUN #{run_count}  |  {now.strftime('%Y-%m-%d  %H:%M:%S')} IST")
```

```
        print(f"  RUN #{run_count}  |  {now.strftime('%Y-%m-%d  %H:%M:%S')} IST")
        print(f"  Market  :  {msg}")
        print(f"{'━'*52}")

        try:
            # ── Step 1: Fetch ──────────────────────────────
            print("  [1/4] Fetching JIOFIN data...")
            df = fetch_jiofin_data(period="1y", interval="1d")
            if df is None:
                print("  ⚠️  No data — will retry next interval")
                return "SKIP"

            # ── Step 2: SQL ────────────────────────────────
            print("  [2/4] Saving to SQL database...")
            upsert_to_sql(df, engine)

            # ── Step 3: Google Sheets ──────────────────────
            print("  [3/4] Pushing to Google Sheets...")
            push_to_google_sheets(engine, gc, SHEET_NAME)

            # ── Step 4: Timestamp ──────────────────────────
            print("  [4/4] Writing timestamp...")
            write_timestamp(gc, SHEET_NAME, df, run_count)

            # ── Summary ────────────────────────────────────
            print(f"\n  ✅ Run #{run_count} complete!")
            print(f"  🗓️ Latest date  : {df['date'].max()}")
            print(f"  💰 Latest close : ₹{df['close_price'].iloc[-1]:.2f}")
            print(f"  📊 Total rows   : {len(df)}")
            return "SUCCESS"

        except Exception as e:
            print(f"  ❌ Error in run #{run_count}: {e}")
            return "ERROR"

    # ── Run history tracker ────────────────────────────────
    history = []

    # ═══════════════════════════════════════════════════════
    #   AUTO-UPDATE LOOP — RUNS UNTIL YOU PRESS STOP
    #   Press the STOP (■) button in Colab to stop
    # ═══════════════════════════════════════════════════════
    print("🚀 JIOFIN AUTO-UPDATE PIPELINE STARTED")
    print(f"⏱️  Update interval : every {INTERVAL_SEC} seconds ({INTERVAL_SEC//60} min)")
    print("🔴  To stop         : press the STOP button (■) next to the cell")
    print("━"*52)

    try:
        while True:
            status = run_once()

            # Log run history
            history.append({
                "run"   : run_count,
                "time"  : datetime.now(IST).strftime('%H:%M:%S'),
                "status": status
            })
```

```
            # Print last 5 runs summary
            print(f"\n  📋 Run history (last 5):")
            print(f"  {'Run':<6} {'Time':<12} {'Status'}")
            print(f"  {'─'*30}")
            for h in history[-5:]:
                icon = "✅" if h['status'] == "SUCCESS" else "⚠️ " if h['status'] == "SKIP" else "❌"
                print(f"  #{h['run']:<5} {h['time']:<12} {icon} {h['status']}")

            print(f"\n  ⏳ Next run in {INTERVAL_SEC//60} min(s)...")
            print(f"  🔴 Press STOP button to stop the pipeline")

            time.sleep(INTERVAL_SEC)

    except KeyboardInterrupt:
        print(f"\n\n{'━'*52}")
        print(f"  ⛔ Pipeline stopped manually")
        print(f"  Total runs completed : {run_count}")
        print(f"  Stopped at          : {datetime.now(IST).strftime('%Y-%m-%d %H:%M:%S')} IST")
        print(f"{'━'*52}")
```

```
#34    22:27:14      ✅ SUCCESS
#35    22:32:17      ✅ SUCCESS

⏳ Next run in 5 min(s)...
🔴 Press STOP button to stop the pipeline


RUN #36  |  2026-04-25  22:37:17 IST
Market   :  Weekend — NSE closed

[1/4] Fetching JIOFIN data...
📡 Fetching JIOFIN.NS data...
✅ Fetched 248 rows
🗓️ Latest date  : 2026-04-24
💰 Latest close : ₹245.73
[2/4] Saving to SQL database...
✅ SQL updated — 249 total rows in database
[3/4] Pushing to Google Sheets...
✅ Daily_Data tab — 249 rows written
```